

DBMS LAB

EXPERIMENT – 4

OBJECTIVE

To study and implement SQL operators and aggregate functions such as MIN(), MAX(), SUM(), COUNT(), and AVG() for retrieving and analyzing data from database tables.

Topic: SQL Operators and Aggregate Functions

MIN • MAX • SUM • COUNT • AVG • WHERE • LIKE • BETWEEN • IN • AND • OR • NOT • ORDER BY • GROUP BY
• Subqueries • Dates

1. Introduction

In SQL, **operators** and **aggregate functions** are used to retrieve, filter, compare, and analyze data stored in database tables. Operators help specify conditions in a query, while aggregate functions perform calculations on a group of rows and return a single result.

For example, we can use **>** to find doctors whose salary is greater than a particular amount, **LIKE** to find doctors whose ID follows a pattern, and **COUNT()** or **AVG()** to generate summary information.

2. SQL Operators

2.1 Comparison Operators

Operator	Meaning
=	Equal to
<>	Not equal to
>	Greater than
<	Less than
>=	Greater than or equal to
<=	Less than or equal to

Example:

```
SELECT *
FROM DOC_REG
WHERE SALARY > 75000;
```

To find doctors whose qualification is not MBBS:

```
SELECT *
FROM DOC_REG
WHERE QUALIFICATION <> 'MBBS';
```

2.2 Logical Operators

- **AND** – all specified conditions must be true.
- **OR** – at least one specified condition must be true.
- **NOT** – reverses a condition.

```
SELECT *
FROM DOC_REG
WHERE SALARY > 75000 AND SALARY < 100000;
```

2.3 LIKE Operator

The **LIKE** operator is used for pattern matching. % represents zero or more characters, while _ represents exactly one character.

```
SELECT *
FROM DOC_REG
WHERE DOC_ID LIKE 'DC%';
```

2.4 BETWEEN Operator

The **BETWEEN** operator selects values within a specified range, generally including both limits.

```
SELECT *
FROM DOC_REG
WHERE SALARY BETWEEN 75000 AND 100000;
```

2.5 IN Operator

The **IN** operator compares a value with a list of specified values.

```
SELECT *
FROM DOCTOR
WHERE DEPARTMENT IN ('Cardiac', 'ENT', 'Orthopaedic');
```

3. Aggregate Functions

Aggregate functions perform calculations on a set of rows and return a single value. The commonly used functions are **MIN()**, **MAX()**, **SUM()**, **COUNT()**, and **AVG()**.

MIN()

Returns the smallest value.

```
SELECT MIN(SALARY)
FROM DOC_REG;
```

MAX()

Returns the largest value.

```
SELECT MAX(SALARY)
FROM DOC_REG;
```

SUM()

Calculates the total of numeric values.

```
SELECT SUM(SALARY)
FROM DOC_REG;
```

COUNT()

Counts rows or non-NULL values.

```
SELECT COUNT(*)
FROM PATIENT;
```

AVG()

Calculates the average of numeric values.

```
SELECT AVG(SALARY)
FROM DOC_REG;
```

4. Aggregate Functions with WHERE

Aggregate functions can be combined with the **WHERE** clause to calculate a result only for records satisfying a condition.

```
SELECT AVG(SALARY)
FROM DOC_REG
WHERE QUALIFICATION = 'MBBS';
```

5. GROUP BY Clause

The **GROUP BY** clause divides rows into groups based on one or more columns. Aggregate functions can then be applied separately to each group.

```
SELECT DEPARTMENT, AVG(SALARY), MAX(SALARY),
MIN(SALARY), COUNT(*)
FROM DOC_REG
GROUP BY DEPARTMENT;
```

6. Subquery

A **subquery** is a query written inside another SQL query. The result of the inner query is used by the outer query.

```
SELECT *
FROM DOC_ON_CALL
WHERE FEE_PER_CALL =
(SELECT MAX(FEE_PER_CALL)
FROM DOC_ON_CALL);
```

7. ORDER BY and FETCH FIRST

The **ORDER BY** clause arranges the result in ascending or descending order. **FETCH FIRST** limits the number of rows returned.

```
SELECT *
FROM DOC_REG
ORDER BY SALARY DESC
FETCH FIRST 5 ROWS ONLY;
```

8. Easy Tricks for Finding Second Highest and Second Smallest

These questions are very common in DBMS practicals. The easiest way to understand them is: **first find the highest/lowest value, then find the next value.**

8.1 Method 1 – Using MAX() for Second Highest

Suppose the salaries are: **50,000, 60,000, 75,000, 90,000**. The highest salary is 90,000. To find the second highest, find the maximum salary that is **less than 90,000**.

```
SELECT MAX(SALARY) AS SECOND_HIGHEST
FROM DOC_REG
WHERE SALARY < (SELECT MAX(SALARY) FROM DOC_REG);
```

EASY TRICK

Second Highest = MAX(value) where value is less than the highest value.

8.2 Method 2 – Using MIN() for Second Smallest

The same idea works in the opposite direction. First find the smallest value. Then find the minimum value that is **greater than** that smallest value.

```
SELECT MIN(SALARY) AS SECOND_SMALLEST
FROM DOC_REG
WHERE SALARY > (SELECT MIN(SALARY) FROM DOC_REG);
```

EASY TRICK

Second Smallest = MIN(value) where value is greater than the smallest value.

8.3 Method 3 – Sort the Values

Another simple approach is to sort the values. For the second highest, sort from **highest to lowest** and take the second value. For the second smallest, sort from **lowest to highest** and take the second value.

```
SELECT SALARY
FROM DOC_REG
ORDER BY SALARY DESC
OFFSET 1 ROW FETCH NEXT 1 ROW ONLY;
```

For the second smallest, change **DESC** to **ASC**.

```
SELECT SALARY
FROM DOC_REG
ORDER BY SALARY ASC
OFFSET 1 ROW FETCH NEXT 1 ROW ONLY;
```

8.4 Method 4 – Using DISTINCT with ORDER BY

If duplicate values are present, you usually want the **second different (distinct) value**. In that case, use DISTINCT before sorting.

```
SELECT SALARY
FROM (SELECT DISTINCT SALARY FROM DOC_REG)
ORDER BY SALARY DESC
```

```
FETCH FIRST 2 ROWS ONLY;
```

The second row of this result is the second-highest **distinct** salary. For the second smallest, use **ORDER BY SALARY ASC**.

8.5 Method 5 – Using DENSE_RANK()

For more advanced SQL, **DENSE_RANK()** is useful when duplicate values exist. It gives the same rank to equal values and does not skip the next rank.

```
SELECT SALARY
FROM (
  SELECT SALARY,
  DENSE_RANK() OVER (ORDER BY SALARY DESC) AS R
  FROM DOC_REG
)
WHERE R = 2;
```

Here, **R = 2** means the second-highest distinct salary. This method is especially useful when more than one doctor has the same second-highest salary.

WHAT SHOULD STUDENTS REMEMBER?

For basic practical questions, the **MAX() + subquery** method is usually the easiest to remember. If duplicate values matter, **DENSE_RANK()** is a better method.

9. Dates in SQL – Very Important

Students often get confused when a question says “**between 1 June and 1 July**”. The main reason is that a database date can also contain a **time**.

9.1 A Date Can Have a Time

In Oracle, a DATE value can contain both the date and the time. For example:

```
01-JUL-2016 10:30 AM
01-JUL-2016 04:15 PM
01-JUL-2016 11:59 PM
```

All three records are on **1 July**, but their times are different.

9.2 What Does TO_DATE('01-JUL-2016', ...) Mean?

When we write a date using TO_DATE() without specifying a time, Oracle uses **12:00 AM** (midnight) as the time.

```
TO_DATE('01-JUL-2016', 'DD-MON-YYYY')
```

So this represents:

```
01-JUL-2016 12:00 AM
```

REMEMBER THIS

12 AM = Midnight = Start of the day

12 PM = Noon = Middle of the day

9.3 Does BETWEEN Include the Two Dates?

Yes. **BETWEEN** includes both the starting value and the ending value.

```
WHERE ADMT_ON BETWEEN  
TO_DATE('01-JUN-2016','DD-MON-YYYY')  
AND TO_DATE('01-JUL-2016','DD-MON-YYYY')
```

The important point is that the ending value here is **1 July at 12:00 AM**. Therefore, a record such as **1 July 2016 at 10:30 AM** is NOT included.

VERY IMPORTANT

BETWEEN is inclusive, but the time still matters.

1 July 12:00 AM is included.

1 July 10:30 AM is not included when the upper limit is exactly 1 July 12:00 AM.

9.4 Then How Do We Include the COMPLETE End Date?

If the question says “**from 1 June 2016 to 1 July 2016, including the complete day of 1 July**”, the safest method is to start from 1 June and stop **before 2 July**.

```
SELECT *  
FROM PAT_ADMIT  
WHERE ADMT_ON >= TO_DATE('01-JUN-2016','DD-MON-YYYY')  
AND ADMT_ON < TO_DATE('02-JUL-2016','DD-MON-YYYY');
```

Why 2 July? Because this condition includes everything from the beginning of 1 June up to, but not including, the beginning of 2 July. Therefore, **the entire day of 1 July is included**.

9.5 The Easy Date-Range Formula

LEARN THIS PATTERN

Start Date → >=

Next Day After End Date → <

Example: 1 June to 1 July inclusive

>= 1 June AND < 2 July

9.6 What About 12 AM and 12 PM?

Students commonly confuse these two:

Time	Meaning	Think of it as
12:00 AM	Midnight	Beginning of the day
12:00 PM	Noon	Middle of the day
11:59 PM	Almost midnight	End of the day

So, if a question says “**up to 1 July**” and you want the complete 1 July, do not simply use 1 July at 12:00 AM as the upper limit. Use **2 July as the exclusive upper limit**.

9.7 Another Option – TRUNC()

If we only care about the calendar date and want to ignore the time part, Oracle's **TRUNC()** can be used.

```
SELECT *
FROM PAT_ADMIT
WHERE TRUNC(ADMT_ON) BETWEEN
TO_DATE('01-JUN-2016','DD-MON-YYYY')
AND TO_DATE('01-JUL-2016','DD-MON-YYYY');
```

This treats the values as dates without considering their time portion. It is easy to understand, but for large tables, the **>= start AND < next day** approach is generally preferable because it can be more index-friendly.

9.8 Quick Examples

Question: Records from 1 June only, including the whole day?

```
WHERE ADMT_ON >= TO_DATE('01-JUN-2016','DD-MON-YYYY')
AND ADMT_ON < TO_DATE('02-JUN-2016','DD-MON-YYYY')
```

Question: Records from 1 June through 3 June, including all of 3 June?

```
WHERE ADMT_ON >= TO_DATE('01-JUN-2016','DD-MON-YYYY')
AND ADMT_ON < TO_DATE('04-JUN-2016','DD-MON-YYYY')
```

ONE-LINE MEMORY TRICK

Want the whole end date? Go one day ahead and use <.

10. Practical Questions

Write and execute SQL queries for the following questions. Do not copy the answers. Verify the output after executing each query.

Q1. Calculate the total number of patients in the Cardiac department.

- Q2. Calculate the average salary of all regular doctors whose qualification is either MD or MBBS.
- Q3. Find the details of doctors whose ID starts with DC.
- Q4. Find the details of all doctors who are NOT MBBS.
- Q5. Find the details of patients whose age is greater than 45.
- Q6. Find the details of the doctor(s)-on-call having the maximum fee per call.
- Q7. Find the details of patients who are being treated for a Neuro Problem.
- Q8. Find the total number of patients visiting for General Surgery.
- Q9. Calculate the average payment made by patients admitted to the hospital.
- Q10. Find the details of rooms that are General rooms without AC.
- Q11. Find the names and fee per call of doctors-on-call whose fee is above ₹1000.
- Q12. Find the details of the top 5 highest-paid doctors.
- Q13. Find the details of doctors whose salary is greater than ₹75,000 and less than ₹1,00,000.
- Q14. Find the list of patients admitted to the hospital between 1 June 2016 and 3 June 2016.
- Q15. Find the second-highest salary of an MD doctor in the Orthopaedic department.
- Q16. Find the list of vacant rooms that are Private rooms with AC facility.
- Q17. Find the count of female patients who are not diagnosed with a surgery-related problem.
- Q18. Find the count of ENT patients in the hospital for the year 2016.
- Q19. Find the total number of regular doctors who belong to the Cardiac, ENT, and Orthopaedic departments and have an MD qualification.

11. Functions and Operators to Practice

Aggregate Functions	Operators / Clauses	Other Concepts
MIN()	=	Subqueries
MAX()	<>	Date functions
SUM()	< / > / <= / >=	TO_DATE()

COUNT()	AND / OR / NOT	EXTRACT()
AVG()	LIKE / BETWEEN / IN	TRUNC()
	WHERE / ORDER BY	Date + Time
	GROUP BY / FETCH FIRST	12 AM / 12 PM

IMPORTANT

Students must write the SQL query for every question, execute it, and verify the output. Understand why a particular operator, aggregate function, clause, subquery, or date condition is being used.